# Guide complet : méthodologie et interface

## Détection automatique des crises d'absence à partir de signaux EEG

Projet Data Mining - 3AINFO

Comment tout a été construit, étape par étape, avec le **quoi** → **comment** → **pourquoi** de chaque décision


**0. Comment lire ce guide** 3

**A. Architecture du projet** 4

A.1 Les fichiers et leur rôle . . . 4

A.2 Le flux de données, de bout en bout . . . 4

A.3 Pourquoi séparer src/ (calcul) et app.py (affichage) . . . 5

**B. Méthodologie en détail** 5

B.1 Les données et leur conversion en CSV . . . 5

B.2 Les annotations (vérité terrain) . . . 6

B.3 Le prétraitement . . . 6

B.4 Étiquetage et découpage en fenêtres . . . 7

B.5 Les features (caractéristiques) . . . 7

B.6 La sélection des meilleures features . . . 8

B.7 Les quatre modèles . . . 9

B.8 L'évaluation . . . 10

B.9 La détection temporelle : recoller les intervalles . . . 12

**C. L'interface Streamlit en détail** 13

C.1 Comment fonctionne Streamlit (à savoir avant tout) . . . 13

C.2 Organisation du code de l'interface . . . 13

C.3 L'identité visuelle . . . 14

C.4 La navigation et l'état . . . 15

C.5 Comment l'interface se connecte au modèle . . . 15

C.6 Les 7 pages, une par une . . . 16
C.7 Les choix techniques et leurs raisons . . . 18
C.8 Comment l'interface a été vérifiée . . . 18
C.9 Limites de l'interface . . . 19
**D. Résultats clés (à retenir) 19**
**E. Points d'attention, limites et améliorations 20**
E.1 À assumer avec confiance (limites connues) . . . 20
E.2 Améliorations possibles (à présenter comme perspectives) . . . 20
E.3 Questions difficiles et réponses solides . . . 21
**F. Glossaire et commandes 22**
F.1 Glossaire . . . 22
F.2 Commandes utiles . . . 22

# 0. Comment lire ce guide

Ce document explique **tout le projet** : les choix de méthode (partie B) et la construction de l'interface Streamlit (partie C). Pour chaque décision, il suit le même schéma : **quoi** (ce qui est fait), **comment** (la technique, avec le code quand il aide) et **pourquoi** (la raison, et l'alternative écartée).

Les termes techniques sont expliqués à leur première apparition, avec une analogie. Un **glossaire** se trouve à la fin (partie F). Les passages encadrés en orange (**points d'attention**) signalent ce qui est une limite ou ce qui n'a pas été vérifié : ils sont là pour que vous puissiez présenter le projet avec assurance, sans être pris en défaut.

| Partie | Contenu |
|---|---|
| **A** | Architecture : les fichiers, le flux de données, la séparation calcul / affichage |
| **B** | Méthodologie : données, CSV, prétraitement, fenêtres, features, sélection, 4 modèles, évaluation, détection temporelle |
| **C** | Interface : comment fonctionne Streamlit, l'identité visuelle, la navigation par étapes, la connexion aux modèles, les 7 pages une par une, les choix techniques, les vérifications |
| **D** | Résultats clés |
| **E** | Points d'attention, limites et améliorations |
| **F** | Glossaire et commandes utiles |

# A. Architecture du projet

## A.1 Les fichiers et leur rôle

| Fichier | Rôle |
|---|---|
| `app.py` (699 lignes) | L'interface Streamlit : navigation par étapes, 7 pages, patient par patient. Ne contient aucun calcul scientifique. |
| `app_questions.py` | Ancienne interface organisée selon les questions Q1 à Q14 du sujet (utile pour le rapport). |
| `ui/theme.py` (297 lignes), `ui/components.py` (141 lignes) | Identité visuelle : palette, CSS, gabarit des graphiques ; composants réutilisables (navigation, en-tête, panneaux, messages, légendes). `.streamlit/config.toml` : thème de base. |
| `src/convert_to_csv.py` | Conversion sans perte des `.mat` et de l'Excel en CSV, avec vérification de l'égalité exacte. |
| `src/data_loader.py` | Lecture des CSV et des annotations (corrections des doublons et de la crise aberrante). |
| `src/preprocessing.py` | Choix des canaux, filtre, normalisation, étiquettes, découpage en fenêtres. |
| `src/features.py` | Calcul des 28 features par canal (temporelles et fréquentielles). |
| `src/dataset.py` | Assemble le dataset : une ligne = une fenêtre, pour tous les patients. |
| `src/selection.py` | Classement des features par 4 critères et suppression des redondances. |
| `src/models.py` | Les 4 modèles, les deux types de séparation train/test, les métriques, le leave-one-patient-out. |
| `src/detection.py` | Transforme les prédictions par fenêtre en intervalles de crise et les compare aux annotations. |
| `src/evaluate.py`, `results/` | Évaluation de référence (leave-one-patient-out, détection stricte) : **source unique** des chiffres de l'app et des documents. |
| `data_csv/`, `cache/` | Données converties et caches. Non versionnés dans git (données médicales, fichiers trop gros). |
| `README.md`, `requirements.txt`, `.gitignore` | Lancement, dépendances, fichiers exclus du dépôt GitHub. |

## A.2 Le flux de données, de bout en bout

| Étape | Entrée → sortie |
|---|---|
| 1. Conversion (une seule fois) | 22 fichiers `.mat` + Excel → `data_csv/*.csv` (identiques bit à bit) |
| 2. Lecture | CSV → matrice `[échantillons × canaux]` en float64, à 256 Hz |
| 3. Prétraitement | Signal brut → filtré 0,5-40 Hz → normalisé (médiane / IQR) par canal |
| 4. Étiquetage | Crises annotées (secondes) → vecteur de 0 / 1 par échantillon |
| 5. Fenêtrage | Signal → fenêtres de 2 s décalées de 1 s ; classe 1 si au moins 50 % de crise |
| 6. Features | Chaque fenêtre → 56 nombres (28 features × moyenne et max sur les canaux) |
| 7. Dataset | 33 206 lignes × 62 colonnes (56 features + méta-informations + classes) |
| 8. Sélection | 56 features → 15 meilleures, non redondantes |
| 9. Modèles | 15 features → probabilité de crise par fenêtre (DT, RF, KNN, SVM) |
| 10. Détection temporelle | Probabilités → seuil → lissage → fusion → intervalles [début, fin] |
| 11. Interface | Affiche chaque étape, patient par patient, de façon interactive |

### A.3 Pourquoi séparer `src/` (calcul) et `app.py` (affichage)

**Quoi** : toute la logique de traitement est dans `src/`, l'interface ne fait que l'appeler et l'afficher. **Pourquoi** : *analogie du restaurant : la cuisine (src) prépare, la salle (app) sert.* Concrètement : (1) on peut tester les calculs sans lancer l'interface ; (2) les deux interfaces (`app.py` et `app_questions.py`) réutilisent exactement le même code, donc donnent les mêmes résultats ; (3) on peut en tirer un notebook ou un script sans rien réécrire.

> *Point d'attention : le sujet demande aussi un **notebook Python** et un **rapport** (section « Travail demandé »). J'ai livré une application Streamlit (votre choix) et un document de méthodologie, mais **pas de notebook**. Comme la logique est dans `src/`, en produire un est rapide si votre encadrant l'exige.*

# B. Méthodologie en détail

## B.1 Les données et leur conversion en CSV

**Quoi**

22 enregistrements EEG (de 20 à 60 minutes, 9,2 heures au total) à **256 Hz** : 256 mesures par seconde et par électrode. Chaque enregistrement contient jusqu'à 19 canaux EEG du système international 10-20 (Fp1, Fp2, F7, F3, Fz, F4, F8, T3, C3, Cz, C4, T4, T5, P3, Pz, P4, T6, O1, O2) plus des canaux non cérébraux (ECG = cœur, EMG = muscles, SLI = stimulation lumineuse). Un fichier Excel d'annotations donne le début et la fin de chaque crise.

**Comment**

- Les fichiers `.mat` sont des `timetable` MATLAB (format MCOS) que `scipy` ne lit pas : j'utilise la bibliothèque `mat-io`, **une seule fois**, pour la conversion.
- Chaque ligne de la table d'origine représente 1 seconde et contient 256 échantillons par canal ; je les aplatis en un signal continu (une ligne = un échantillon), comme le demande le sujet : colonnes `Time` + un canal par colonne.
- Les CSV sont écrits en float64 (pleine précision) avec `pyarrow`, puis **relus et comparés** aux `.mat` d'origine : égalité exacte des valeurs, des noms de colonnes et de l'axe du temps pour les 22 fichiers.
- Les canaux ECG, EMG et SLI sont **conservés** dans les CSV pour ne rien perdre, mais **non utilisés** par les modèles.

**Pourquoi**

- CSV : demandé (« travailler avec CSV sans rien perdre ») et lisible par tout outil. *Analogie : un tableau Excel simple plutôt qu'un format propriétaire.*
- Float64 : mon premier cache était en float32 (moins précis). Après reconstruction depuis les CSV, les features sont quasi identiques (corrélation supérieure à 0,999999998) et les résultats n'ont bougé que de 0,001 à 0,004.
- Canaux EEG seulement : l'ECG est cardiaque, l'EMG musculaire, SLI est le signal de la stimulation lumineuse. Les donner au modèle risquerait de lui faire apprendre un artefact ou la stimulation plutôt que la crise.
- 21 fichiers sur 22 ont les 19 canaux EEG ; **200625A_F n'en a que 8**. Chaque enregistrement utilise tous ses canaux EEG disponibles. C'est possible parce que les features sont ensuite agrégées sur les canaux (section B.5).

> *Seule perte connue : dans les marqueurs textuels du fichier `*_h.mat` (« Début »…), les accents sont déjà abîmés à la lecture par la bibliothèque. Ces textes sont informatifs et ne servent pas au pipeline.*

## B.2 Les annotations (vérité terrain)

### Quoi → comment → pourquoi

- **Les temps de l'Excel sont des fractions de jour** (heure de la journée). Une durée de 0,00008 correspond à 7 s. Début relatif de la crise = (DebCE − DebutTrace) × 86 400 s. *Validation : pour 190304A_E, la formule donne 1 291 s, ce qui coïncide avec le marqueur « crise » à 21:32 du fichier* `h`.
- **Doublons** : 230406B-A et 230515B-G apparaissent deux fois. Je garde la dernière ligne (le fichier s'appelle « MAJ »). Choix à confirmer avec l'encadrant.
- **Crise aberrante** : pour 210504B-C, la fin de la crise 1 donne 6 heures dans un enregistrement de 52 minutes. Règle automatique : on écarte toute crise de durée ≤ 0, supérieure à 120 s ou de début négatif.
- Résultat : **82 crises valides** dans 19 enregistrements, durée moyenne 13,2 s (min 2 s, max 60 s). Trois enregistrements n'ont aucune crise (200625A_F, 210204B_C, 210208B_G) : conservés comme exemples négatifs.

### Étiquettes probablement incomplètes (constat après vérification)

- L'enregistreur pose des marqueurs « absence » ou « crise » (fichiers `_events.csv`). Sur 23 marqueurs, **17 tombent à ± 3 s d'une crise annotée**, toujours 1 à 13 s **après** le début annoté : l'alignement des annotations est donc validé sur 16 enregistrements (et pas seulement un).
- **6 marqueurs n'ont aucune crise annotée** : 210208B_G (3 marqueurs, **aucune crise dans l'Excel**, utilisé jusque-là comme exemple négatif) et 210427B_C (3 marqueurs pour une seule crise annotée).
- Le signal de 210208B_G montre trois salves d'oscillations amples et régulières de 7 à 15 s (vers 572, 824 et 1087 s), avec une puissance relative à 3 Hz de 0,55 à 0,60 (0,16 en temps normal, 0,44 dans les crises annotées). Le SVM les retrouve toutes, le Random Forest 2 sur 3 (figure : `audit_210208B_G_marqueurs.png`).

> *Conclusion prudente : ce sont **probablement des crises non annotées**. Les « fausses alarmes » de ces fichiers sont donc peut-être de bonnes détections, et la précision est sous-estimée. Seul un expert peut le confirmer : **je n'ai rien relabellisé**. Décision à prendre avec l'encadrant (ajouter ces crises ou exclure ces deux enregistrements), puis relancer* `python -m src.evaluate`.

## B.3 Le prétraitement

*Analogie : avant de peser un ingrédient, on retire l'emballage et on règle la balance à zéro. Le signal brut contient des choses sans rapport avec la crise, il faut les enlever avant d'analyser.*

### Étape 1 : filtre passe-bande 0,5 à 40 Hz

```
sos = butter(order, [lo, hi], btype="band", fs=fs, output="sos")   # Butterworth d'ordre 4
return sosfiltfilt(sos, x, axis=0)                                  # aller-retour : phase nulle
```

- **Quoi** : un filtre qui ne laisse passer que les fréquences entre 0,5 et 40 Hz.
- **Pourquoi 0,5 Hz** : retire la dérive très lente (mouvements, sueur, électrodes) qui domine l'amplitude brute. **Pourquoi 40 Hz** : supprime le bruit du secteur à 50 Hz et l'activité musculaire. Le rythme des absences (environ 3 Hz) reste dans la bande.
- **Pourquoi « aller-retour »** (`sosfiltfilt`) : un filtre classique décale le signal dans le temps. Comme on veut localiser début et fin de crise à la seconde près, aucun décalage n'est acceptable.
- **Pourquoi Butterworth** : réponse la plus plate dans la bande passante (pas d'ondulation qui déformerait l'amplitude). **Forme SOS** : numériquement stable.

### Étape 2 : normalisation robuste par canal et par enregistrement

```
med = np.median(x, axis=0)
q1, q3 = np.percentile(x, [25, 75], axis=0)
return (x - med) / np.maximum(q3 - q1, 1e-9)          # (x - médiane) / IQR
```

- **Quoi** : on soustrait la médiane et on divise par l'écart interquartile (IQR = Q3 − Q1) de chaque canal.

- **Pourquoi** : les amplitudes varient beaucoup d'un enfant et d'une session à l'autre (impédances d'électrodes de 15 kΩ à 90 kΩ dans les marqueurs, âges de 5 à 25 ans). Sans normalisation, le modèle apprendrait « quel enfant » plutôt que « quelle crise ».
- **Pourquoi médiane / IQR plutôt que moyenne / écart-type** : ils sont insensibles aux artefacts et aux crises elles-mêmes (moins de 5 % du temps). *Analogie : le salaire médian ne bouge pas si un milliardaire entre dans la pièce, la moyenne si.*

*Points d'attention : (1) la normalisation utilise tout l'enregistrement ; en temps réel, il faudrait une ligne de base glissante calculée sur le passé. (2) Aucun retrait d'artefacts (ICA, rejet de fenêtres) : les artefacts restent un problème réel, visible dans les faux positifs.*

## B.4 Étiquetage et découpage en fenêtres

### Étiquette par échantillon

Un vecteur `y` de même longueur que le signal : 1 entre le début et la fin de chaque crise annotée, 0 ailleurs.

### Fenêtres

```
w    = int(win_s * fs)                        # 2 s * 256 Hz = 512 échantillons
step = max(1, int(w * (1 - overlap)))         # recouvrement 50 % -> pas de 256 = 1 s
W    = sliding_window_view(x, w, axis=0)[::step]            # toutes les fenêtres d'un coup
frac = sliding_window_view(y.astype(np.float32), w)[::step].mean(axis=1)   # part en crise
Class = (frac >= 0.5)                         # fenêtre = crise si au moins 50 % en crise
```

| Paramètre | Valeur | Pourquoi |
|---|---|---|
| Pourquoi découper | - | Un point isolé ne dit rien (un rythme se voit sur plusieurs cycles) ; le modèle a besoin d'entrées de taille fixe ; l'EEG est quasi stationnaire sur quelques secondes. *Analogie : on lit des mots, pas des lettres.* |
| Taille | 2 s = 512 échantillons | 6 cycles d'une pointe-onde à 3 Hz ; résolution en fréquence = 1/T = 0,5 Hz, assez fine pour séparer Delta de Theta et isoler la bande 3 Hz. 1 s : 1 Hz de résolution (trop grossier). 4 s : dilue les crises courtes. |
| Recouvrement | 50 % (pas de 1 s) | Une décision par seconde ; une crise n'est pas coupée en deux fenêtres inexploitables. Valeur du sujet. |
| Règle de classe | au moins 50 % en crise | Règle du sujet : une fenêtre à cheval sur le début est « crise » si au moins une seconde sur deux est ictale. |

Résultat : **33 206 fenêtres**, dont **1 168 de crise (3,5 %)**. Le déséquilibre est d'environ 1 pour 27 : *3 pièces fausses dans un sac de 100*. Une 3e colonne `Class3` marque aussi les fenêtres « pré-crise » (les 10 s avant le début : 806 fenêtres), préparée pour la prévention.

*Points d'attention : (1) **je n'ai pas comparé expérimentalement 1 s, 2 s et 4 s** : le choix de 2 s repose sur le raisonnement ci-dessus et sur la suggestion du sujet ; l'interface permet de les tester mais je n'en ai pas évalué les performances. (2) Avec 50 % de recouvrement, deux fenêtres voisines partagent la moitié de leurs échantillons : un split aléatoire met des fenêtres quasi identiques en train et en test (**fuite d'information**), d'où le split par patient en B.8.*

## B.5 Les features (caractéristiques)

Une **feature** est un nombre qui résume une fenêtre. *Analogie : pour décrire une personne à un enquêteur, on donne la taille, le poids, l'âge. Le modèle ne voit pas les 512 valeurs brutes, il voit ces résumés.* Pour chaque fenêtre et chaque canal, on calcule 28 features, puis on agrège sur les canaux par la **moyenne** et le **maximum** : 28 × 2 = **56 features**.

### Comment on a trouvé les candidates

En partant de ce que **l'expert voit** sur un EEG de crise et en le traduisant en nombre :

| Ce qu'on observe | Nombre qui le mesure |
|---|---|
| Le signal est plus **ample** | Std, Variance, Min, Max, Amplitude (max − min), RMS, Energy (Σ x² / fs) |
| Il est **pointu / agité** | LineLength = Σ \|x(t+1) − x(t)\| (longueur de la courbe) |
| Il est **rythmique** | ZeroCross (passages par zéro par seconde), DomFreq (fréquence du pic spectral) |
| Il a une forme particulière | Hjorth mobilité et complexité, Skewness (asymétrie), Kurtosis (aplatissement) |
| L'énergie se concentre vers **3 Hz** | Puissance de la bande 2,5-4,5 Hz (bande que j'ai ajoutée), puissances Delta, Theta, Alpha, Beta (bandes du sujet) |
| Le spectre est concentré | SpecEntropy (entropie du spectre), SEF95 (fréquence sous laquelle se trouvent 95 % de la puissance), ratio Theta/Alpha |

### Le calcul fréquentiel : la FFT

La **FFT** (transformée de Fourier rapide) décompose la fenêtre en sinusoïdes et indique **quelle énergie** se trouve à chaque fréquence. *Analogie : un prisme qui sépare la lumière blanche en couleurs, ou un accord de musique décomposé en notes.* On multiplie d'abord la fenêtre par une **fenêtre de Hann** (qui adoucit les bords ; sans elle, les bords créent de fausses fréquences). La **puissance de bande** est la somme de l'énergie des fréquences de la bande ; la **puissance relative** est cette puissance divisée par la puissance totale 0,5-40 Hz, ce qui la rend indépendante de l'amplitude et donc comparable entre enfants.

### Pourquoi agréger par moyenne et par maximum

- Les absences sont des crises **généralisées** : tous les canaux changent ensemble. La **moyenne** capture ce comportement global.
- Le **maximum** capte un canal qui réagit fortement (crise plus marquée sur une région, ou artefact local).
- Cela rend les features **indépendantes du nombre de canaux** (19 ou 8) et évite 532 features (19 × 28) pour seulement 1 168 fenêtres de crise, ce qui aggraverait le surapprentissage.

## B.6 La sélection des meilleures features

Parmi 56 features, beaucoup sont redondantes ou peu utiles. `src/selection.py` en retient **15** en deux temps.

### Temps 1 : classement par 4 critères, puis rang moyen

| Critère | Ce qu'il mesure | Analogie |
|---|---|---|
| Information mutuelle | De combien l'incertitude sur la classe diminue quand on connaît la feature (capte aussi les liens non linéaires) | Connaître la taille d'une personne aide à deviner si c'est un adulte |
| Importance Random Forest | À quel point les arbres s'appuient sur la feature pour trancher | L'indice que les experts consultent le plus souvent |
| d de Cohen | (moyenne crise − moyenne non-crise) / écart-type typique : l'éloignement des deux groupes | Deux foules dont les tailles moyennes sont très différentes |
| F-ANOVA | Variation entre les classes / variation à l'intérieur des classes | Les groupes diffèrent-ils plus entre eux qu'en leur sein ? |

Pour que les 32 038 négatifs n'écrasent pas les 1 168 positifs, le calcul utilise **toutes les fenêtres de crise et 12 000 fenêtres hors crise** tirées au hasard.

### Temps 2 : suppression des redondances

On parcourt les features par score décroissant et on **écarte toute feature dont la corrélation absolue avec une feature déjà retenue dépasse 0,9**, jusqu'à en avoir 15. Pourquoi : Std, Variance, RMS et Energy mesurent la même chose, en garder une suffit.

**Résultat**

Les 15 retenues : `Std_mean`, `Min_max`, `Rel_Hz3_mean`, `Rel_Hz3_max`, `LineLength_mean`, `ZeroCross_mean`, `ZeroCross_max`, `Power_Hz3_mean`, `Max_max`, `DomFreq_mean`, `Variance_mean`, `LineLength_max`, `Rel_Beta_max`, `Power_Beta_mean`, `HjorthComplexity_max`. **Les features de la bande 3 Hz sont en tête** : cela confirme l'hypothèse médicale du rythme pointe-onde à 3 Hz, et c'est un bon argument à l'oral.

*Point d'attention : le classement est calculé sur **tous** les patients avant l'évaluation leave-one-patient-out. C'est une légère fuite d'information, identique pour les 4 modèles : elle ne fausse pas leur comparaison relative mais rend les scores un peu optimistes.*

## B.7 Les quatre modèles

Un **modèle** est une fonction qui prend les 15 nombres d'une fenêtre et renvoie une **probabilité de crise** entre 0 et 1 (si elle dépasse le seuil, 0,5 par défaut, on dit « crise »). *Analogie : un élève à qui on montre des milliers de fenêtres corrigées et qui en déduit lui-même les règles.* Chaque modèle est un `Pipeline` scikit-learn : **StandardScaler** (met toutes les features à la même échelle : (x − moyenne) / écart-type, appris uniquement sur l'entraînement) puis le classifieur.

**Entraîner** un modèle, c'est ajuster ses paramètres pour **minimiser une fonction de perte** (une mesure de ses erreurs). *Analogie : un professeur qui compte les points perdus ; l'élève change de méthode jusqu'à en perdre le moins possible.* La **pondération des classes** (`class_weight="balanced"`) donne un poids n / (2 × n_classe) : environ **14,2** pour une crise contre **0,52** pour une non-crise, soit une erreur sur une crise qui coûte **27 fois plus**. Sans elle, répondre « jamais de crise » donnerait 96,5 % de bonnes réponses.

| Modèle | Principe (fonction clé) | Réglages | Atouts / limites |
|---|---|---|---|
| **Decision Tree** | Suite de règles « feature > seuil ». À chaque nœud, choisit la règle qui minimise l'**impureté de Gini** = 1 − Σ(proportions²). Exemple : 90 non-crises + 10 crises : Gini = 1 − (0,9² + 0,1²) = 0,18. | `max_depth=8`, `min_samples_leaf=5`, classes pondérées | Lisible. Instable, surapprend. Recall élevé (0,80) mais précision faible (0,33) : beaucoup de fausses alarmes. |
| **Random Forest** | 200 arbres, chacun sur un échantillon tiré au hasard et un sous-ensemble aléatoire de features ; **vote moyen** = probabilité. *200 experts qui n'ont pas vu les mêmes cas.* | `n_estimators=200`, `min_samples_leaf=2` (option : 5), classes pondérées | Variance bien plus faible qu'un seul arbre, gère les features corrélées, donne une probabilité. Moins lisible ; échoue sur 230718A_A (crises de faible amplitude). |
| **KNN** | Pour une fenêtre, trouve les 31 fenêtres connues les plus proches (**distance euclidienne**) et vote, pondéré par la distance. Aucun apprentissage, juste de la comparaison. | `k=31`, `weights="distance"`, **sous-échantillonnage** : 5 non-crises pour 1 crise à l'entraînement | Le KNN n'a pas de pondération de classes : il penchait vers la majorité (k=7 : Recall 0,50). Le sous-échantillonnage et un k plus grand le corrigent (PR-AUC 0,50 → 0,56). |
| **SVM** | Cherche la frontière qui sépare les classes avec la plus grande **marge**. Le **noyau RBF** $K(x, x') = \exp(-\gamma\|x - x'\|^2)$ mesure la similarité et autorise une frontière non linéaire. | RBF, `C=1`, `gamma="scale"`, classes pondérées, `probability=True` | Le plus lent (environ 18 min pour 22 entraînements avec probabilités). **Sa performance dépend beaucoup du seuil** : Recall 0,85 avec la fonction de décision (seuil 0), 0,46 avec la probabilité à 0,5. |

*Points d'attention : **les hyperparamètres n'ont pas été optimisés rigoureusement**. J'ai testé 5 réglages du Random Forest (PR-AUC entre 0,58 et 0,61 : écart non significatif) mais en choisissant sur les mêmes données, donc de façon optimiste ; un réglage honnête demande une validation imbriquée par patient (environ 30 à 40 minutes de calcul). Le StandardScaler est inutile pour les arbres mais*

*inoffensif.*

## B.8 L'évaluation

### Les trois protocoles

| Protocole | Description | Fiabilité |
|---|---|---|
| Split stratifié 80/20 | Séparation aléatoire des fenêtres en gardant la proportion de crises (demandé par le sujet) | **Optimiste** : des fenêtres quasi identiques sont de part et d'autre |
| Split par patient 80/20 | Aucun enregistrement n'est à la fois en train et en test (`GroupShuffleSplit`, on retente des graines jusqu'à avoir plus de 20 crises des deux côtés) | Réaliste, mais un seul tirage (5 enregistrements en test) |
| Leave-one-patient-out | 22 entraînements : chaque enregistrement est testé par un modèle qui ne l'a **jamais vu** | **La plus fiable** : sert à comparer les modèles et alimente l'interface |

### Les métriques : comment les lire

On compare la prédiction à la réalité. La **matrice de confusion** compte 4 cas : **TP** (crise détectée), **TN** (non-crise bien classée), **FP** (fausse alarme), **FN** (crise manquée).

| Métrique | Formule | Question à laquelle elle répond |
|---|---|---|
| Accuracy | (TP + TN) / total | Quelle part de réponses justes ? **Trompeuse ici** : 96,5 % pour « jamais de crise ». |
| **Balanced Accuracy** | (Recall crise + Recall hors-crise) / 2 | Comme l'Accuracy mais chaque classe pèse autant : « jamais de crise » ne fait que **0,50**. **Remplace l'Accuracy** dans l'interface. |
| Précision | TP / (TP + FP) | Quand le modèle crie « crise », a-t-il raison ? (coût des fausses alarmes) |
| **Recall** | TP / (TP + FN) | Quelle part des vraies crises est détectée ? **Priorité** : une crise manquée est le pire cas. |
| F1-score | 2 × P × R / (P + R) | Compromis précision / recall (moyenne harmonique) ; empêche de gagner en criant « crise » partout |
| PR-AUC | aire sous la courbe précision-recall | Qualité du modèle **indépendamment du seuil**, adaptée au déséquilibre |

*Exemple fictif, pour comprendre : sur 100 vraies crises et 900 non-crises, un modèle détecte 80 crises (TP=80, FN=20) et déclenche 40 fausses alarmes (FP=40, TN=860). Recall = 80/100 = 0,80 ; Précision = 80/120 = 0,67 ; F1 = 0,73 ; Accuracy = 940/1000 = 0,94.*

### Comparaison rigoureuse : leave-one-patient-out, 15 features (`python -m src.evaluate`)

| Modèle | AUC | PR-AUC [IC 95 %] | Recall | Précision | F1 [IC 95 %] | F1 moyen / patient |
|---|---|---|---|---|---|---|
| **Random Forest** | 0,942 | 0,590 [0,49-0,79] | 0,574 | 0,724 | 0,640 [0,57-0,74] | 0,666 |
| **KNN (corrigé)** | 0,930 | 0,564 [0,46-0,79] | 0,756 | 0,566 | 0,647 [0,58-0,77] | 0,706 |
| **SVM (probabilité, seuil 0,5)** | 0,927 | 0,544 [0,44-0,71] | 0,457 | 0,608 | 0,522 [0,44-0,61] | 0,497 |
| **Decision Tree** | 0,845 | 0,500 [0,38-0,71] | 0,794 | 0,327 | 0,463 [0,40-0,62] | 0,533 |

Les intervalles de confiance à 95 % viennent d'un bootstrap sur les 19 patients avec crises (300 tirages). Tous les modèles sont évalués au **même seuil de 0,5** sur leur probabilité, avec **exactement** les réglages de `src/models.py` utilisés par l'application.

### Data augmentation, seuil appris et Balanced Accuracy

Deux améliorations de l'évaluation ont été ajoutées. (1) **Data augmentation des crises** (`Augmented` dans `src/models.py`) : pendant l'entraînement seulement, on crée 2 fois plus de fenêtres de crise synthétiques = fenêtre réelle + bruit gaussien (0,1 × l'écart-type de la feature sur les crises). Le test n'est jamais augmenté. (2) **Seuil appris** : au lieu de 0,5, le seuil de chaque modèle maximise le F1 sur 20 % du train (borné à [0,1 ; 0,9]) ; le test n'intervient jamais dans son choix. Raison : à 0,5 chaque modèle a un point de fonctionnement différent (le Decision Tree sur-estime les crises d'un facteur 2,5), donc une part des écarts entre modèles est un artefact du seuil.

| Modèle | F1 strat. base | F1 strat. augm. + seuil | BalAcc strat. base | BalAcc strat. augm. + seuil | F1 patient base | F1 patient augm. + seuil | BalAcc patient base | BalAcc patient augm. + seuil |
|---|---|---|---|---|---|---|---|---|
| **Decision Tree** | 0.54 | 0.70 | 0.91 | 0.88 | 0.46 | 0.63 | 0.87 | 0.84 |
| **Random Forest** | 0.77 | 0.76 | 0.87 | 0.89 | 0.64 | 0.66 | 0.78 | 0.82 |
| **KNN** | 0.69 | 0.72 | 0.90 | 0.86 | 0.65 | 0.64 | 0.87 | 0.85 |
| **SVM** | 0.62 | 0.73 | 0.78 | 0.90 | 0.52 | 0.61 | 0.72 | 0.84 |

*Base = seuil 0,5, sans augmentation. « strat. » = split stratifié 80/20 du sujet (5 découpages, moyenne) ; « patient » = leave-one-patient-out (22 entraînements). Dans le cas « patient », le seuil appris vient des scores des 21 autres enregistrements : c'est légèrement optimiste.*

- **Les quatre modèles se rapprochent** une fois évalués avec augmentation et seuil appris : l'écart entre le meilleur et le moins bon modèle diminue nettement par rapport au seuil 0,5.
- **Le gain de la data augmentation est réel mais modeste, et en partie un effet de seuil** : l'augmentation aide surtout le SVM et le Decision Tree ; avec un seuil appris, un SVM sans augmentation rattrape l'essentiel de l'écart. Le Random Forest ne change pas.
- **Le split stratifié donne des scores plus élevés que le par-patient** (fuite entre fenêtres voisines) : on présente les deux, le stratifié parce que le sujet le demande, le par-patient comme estimation réaliste.

### Verdict

- Le **Decision Tree** est clairement le plus faible : le Random Forest le dépasse en PR-AUC dans 100 % des tirages bootstrap.
- **Random Forest, KNN corrigé et SVM ne sont pas statistiquement distinguables** avec 19 patients : le RF dépasse le KNN dans 65 % des tirages et le SVM dans 90 %, mais les intervalles de confiance se chevauchent largement. Le Random Forest a la meilleure PR-AUC (0,59) et la meilleure précision.
- Les modèles font des **compromis différents** : le RF est sélectif (précision 0,72, Recall 0,57), le KNN corrigé détecte davantage de crises (Recall 0,76) avec plus de fausses alarmes, et son F1 moyen par patient est le plus élevé (0,71). **Recommandation : Random Forest comme modèle principal** (le plus sélectif), KNN corrigé si l'on privilégie de ne rien rater.

*Changement par rapport à une version précédente : le SVM était crédité d'un Recall de 0,85, mais avec un seuil sur la fonction de décision (0). Avec sa probabilité au seuil 0,5, comme les autres modèles, son Recall est de 0,46. Sa PR-AUC, indépendante du seuil, est restée stable (0,553 → 0,544) :* ***c'est le choix du seuil qui change son profil****, pas sa capacité à classer les fenêtres.*

### Expériences complémentaires (ce qui a été essayé, y compris ce qui n'a pas marché)

| Idée testée | Résultat | Décision |
|---|---|---|
| Sous-échantillonner les non-crises (5 pour 1) et k=31 pour le KNN | PR-AUC 0,496 → 0,564 ; F1 moyen par patient 0,623 → 0,706 | **Adopté** (KNN corrigé) |
| 5 réglages du Random Forest (profondeur, feuilles, features par arbre) | PR-AUC entre 0,576 et 0,609 ; meilleur : `min_samples_leaf=5` (0,609) | Écart non significatif, choisi sur les mêmes données : proposé **en option** (curseur de l'app), pas par défaut |
| Normaliser les features par enregistrement (médiane / IQR) | Dégrade tous les modèles (RF : PR-AUC −0,034 ; F1 moyen par patient 0,664 → 0,445) ; ne sauve que partiellement 230718A_A (F1 0 → 0,17) | **Abandonné** |
| Corriger les bornes de ± 1 s (idée : une fenêtre positive dès 1 s de crise) | L'erreur **médiane** des bornes est déjà de 0 s ; la correction l'aggrave (+1 s au début, −1 s à la fin) | **Abandonné** : le biais supposé n'existe pas |

**Pourquoi le Random Forest échoue sur 230718A_A** (0 crise retrouvée sur 7 au début) : ses crises sont de **faible amplitude** (écart-type moyen 1,2 contre 3,5 pour les autres patients) mais avec la puissance relative à 3 Hz la plus forte du jeu (0,60). Le Random Forest s'appuie sur les features d'amplitude absolue et passe à côté ; le SVM voit 77 % de ses fenêtres de crise. **Piste non testée** : un jeu de features sans amplitude absolue (garder puissances relatives, fréquence dominante, Hjorth).

## B.9 La détection temporelle : recoller les intervalles

Le modèle donne une probabilité **par fenêtre** ; on veut « crise de X s à Y s ». Quatre gestes (`src/detection.py`) :

```
p = median_filter(pred, size=smooth)           # 2) lissage : vote majoritaire de 3 fenêtres voisines
for ts, te, v in zip(t_start, t_end, p):
    if v:                                       # fenêtre positive
        if cur and ts - cur[1] <= max_gap:      # 3) fusion : démarre au plus 2 s après la fin
            cur[1] = te                         #    de l'intervalle en cours -> on le prolonge
        else:
            if cur: iv.append(cur)
            cur = [ts, te]                      #    sinon on ouvre un nouvel intervalle
iv = [(a, b) for a, b in iv if b - a >= min_dur]   # 4) on retire ceux de moins de 2 s
```

- **1) Seuil** : probabilité supérieure à 0,5 donne 1, sinon 0. **2) Médian sur 3** : une détection isolée (souvent un artefact) disparaît, un trou d'une fenêtre au milieu d'une crise est comblé. **3) Fusion** : deux détections séparées d'au plus 2 s forment une seule crise. **4) Durée minimale 2 s** : la plus courte crise annotée.
- *Exemple : fenêtres positives débutant à 83, 84, ..., 95 s ; la dernière finit à 97 s : « crise détectée de 83 à 97 s ».*

### Comparaison aux annotations : appariement strict

Chaque intervalle prédit ne compte que pour **une seule** crise réelle (`match_intervals`) : on apparie les paires par recouvrement décroissant, mesuré par l'**IoU** (intersection / union), tant que l'IoU dépasse 0,3. Une crise réelle sans intervalle apparié est **manquée**, un intervalle prédit sans crise réelle est une **fausse alarme** (y compris un doublon ou un intervalle qui couvre plusieurs crises).

| Modèle (seuil 0,5) | Crises retrouvées | Manquées | Fausses alarmes | Par heure | Précision événements |
|---|---|---|---|---|---|
| Random Forest | 68 / 82 (83 %) | 14 | 37 | 4,0 | 0,65 |
| KNN (corrigé) | 77 / 82 (94 %) | 5 | 100 | 10,8 | 0,44 |
| SVM | 56 / 82 (68 %) | 26 | 65 | 7,0 | 0,46 |
| Decision Tree | 75 / 82 (91 %) | 7 | 317 | 34,4 | 0,19 |

**Précision des bornes** (crises retrouvées, Random Forest) : erreur **médiane de 0 s** au début comme à la fin ; erreur moyenne en valeur absolue de 1,25 s au début et 1,84 s à la fin. Les bornes sont donc fiables à environ 1 à 2 s. La mesure large utilisée avant (un intervalle qui chevauche une crise suffit) donnait 90 % pour le Random Forest : elle était **optimiste de 7 points**.

> *Points d'attention : (1) une partie des « fausses alarmes » sont probablement de vraies crises **non annotées** (voir B.2) : la précision réelle est meilleure que celle affichée. (2) Le seuil d'IoU de 0,3 est un choix (réglable dans l'app) ; les résultats changent avec lui. (3) Le Random Forest rate 230718A_A et 220121B_D (cause de la première expliquée en B.8).*

# C. L'interface Streamlit en détail

## C.1 Comment fonctionne Streamlit (à savoir avant tout)

**Streamlit** transforme un script Python en page web. Son principe, essentiel à comprendre : **à chaque interaction (clic, curseur, liste déroulante), le script entier est ré-exécuté de haut en bas.** *Analogie : une feuille de calcul qui recalcule toutes ses cellules à chaque modification.*

Conséquence : sans précaution, un calcul d'une minute se referait à chaque clic. D'où les mécanismes utilisés dans `app.py` :

| Mécanisme | Rôle | Où dans l'app |
|---|---|---|
| `@st.cache_data` | Mémorise le résultat d'une fonction, **clé = ses arguments**. Même arguments : résultat instantané, sans recalcul. Renvoie une copie. | `get_annots`, `get_dataset`, `get_ranking`, `get_loo`, `get_rec_table` |
| `@st.cache_resource` | Idem mais **sans copie** : un seul objet partagé. Adapté aux gros tableaux. | `get_prep` (signaux brut / filtré / normalisé) |
| `st.session_state` | Un dictionnaire qui **survit aux ré-exécutions** : la mémoire de la session. | Étape courante, étapes consultées, réglages, position de la fenêtre, features retenues, résultats d'entraînement |
| `on_click` (rappel) | Fonction exécutée **avant** la ré-exécution du script quand on clique sur un bouton. | Navigation par étapes, boutons Précédente / Suivante des fenêtres |
| `st.query_params` | Lit les paramètres de l'adresse web. | Liens directs : `?step=5&rec=210914B_A` ouvre l'étape 5 sur cet enregistrement |

Les **widgets** (`slider`, `selectbox`, `button`...) renvoient immédiatement leur valeur courante dans le script. Autre particularité utile à connaître : Streamlit **oublie** la valeur d'un widget dès qu'il n'est plus affiché. Sans précaution, revenir sur une étape remettrait tous ses curseurs à zéro (voir C.4).

## C.2 Organisation du code de l'interface

| Fichier | Rôle |
|---|---|
| `app.py` | Orchestration et affichage : chargements en cache, navigation, barre latérale, une fonction par page (`page_overview`, `page_preprocessing`, `page_window`, `page_features`, `page_classification`, `page_detection`, `page_prevention`), puis le routage `PAGES[step]()`. |
| `ui/theme.py` | **Une seule source** pour les couleurs, la typographie, le CSS injecté dans la page et le gabarit Plotly appliqué à tous les graphiques. |
| `ui/components.py` | Composants sans logique scientifique : `stepper`, `step_nav`, `page_header`, `section`, `panel`, `callout`, `chips`, `legend`, `verdict`, `flow`, `status_table`, `brand`, `facts`, `disclaimer`. |
| .streamlit/config.toml | Thème de base de Streamlit (clair, mêmes couleurs que `ui/theme.py`), barre d'outils réduite. |

```
@st.cache_data(show_spinner="Construction du dataset ... ~1 min la première fois")
def get_dataset(win_s, overlap, pre_s):
    return load_or_build_dataset(win_s, overlap, pre_s)    # cache disque géré dans src/dataset.py
```

**Deux niveaux de cache** : la mémoire de Streamlit (instantanée, perdue à l'arrêt du serveur) et un fichier CSV sur disque (survit aux redémarrages ; reconstruit automatiquement si les annotations ont changé). Chaque combinaison fenêtre / recouvrement / zone pré-crise a son propre fichier.

## C.3 L'identité visuelle

**Quoi** : une identité inspirée de l'électrophysiologie (tracés sur papier, équipement hospitalier, publications scientifiques), sobre et scientifique. **Pourquoi** : une interface médicale doit rassurer et rester lisible pendant une démonstration ; le design par défaut de Streamlit (curseurs rouges, listes de boutons radio) ne portait aucune identité.

| Couleur | Valeur | Rôle dans l'application |
|---|---|---|
| Bleu pétrole | `#14505C` (titres : `#0C2F38`) | Navigation active, bouton principal, **prédictions du modèle** |
| Ivoire / surface / ivoire foncé | `#F7F3EA` / `#FBF8F1` / `#EDE7D8` | Fond de page / panneaux et zones de tracé / barre latérale |
| Vert sauge | `#7FA593`, `#4E7566`, `#DCE7DF` | Étiquettes des features retenues, signal normalisé, état « Fait » |
| Cuivre | `#B07A3A` (texte : `#8A5A22`) | Filets de repère, fenêtre courante, probabilité de crise |
| Corail atténué | `#C65D4E`, clair `#F3D5CF` | **Annotations de référence** (crises) et événements critiques uniquement |
| Ambre | `#D8A23E`, clair `#F3E2B8` | Zone pré-critique |
| Gris-bleu | `#8FA6B0` | Phase inter-critique, signal « normal » |
| Gris ardoise | `#58636D` | Texte secondaire, bordures |
| Prune | `#86699A` | Phase post-critique : **réservée**, non affichée (voir ci-dessous) |

- **Une couleur = une signification**, partout dans l'application : corail = annotation de référence, pétrole = prédiction du modèle, ambre = zone pré-critique. Une couleur n'est jamais utilisée seule : chaque graphique a une **légende écrite**, les courbes d'un même graphique se distinguent aussi par leur **style de trait** (plein, tirets, pointillés), et le verdict d'une fenêtre est un **mot** (« Crise manquée ») en plus d'une couleur.
- **Typographie** : titres en serif (Newsreader), texte en IBM Plex Sans, chiffres en IBM Plex Mono (alignement des valeurs). Les polices sont chargées depuis Google Fonts ; sans connexion, des polices système prennent le relais.
- **Sobriété** : pas de dégradés, pas d'ombres, un seul niveau d'encadré (les panneaux de paramètres). Les titres de section sont des filets fins, pas des cartes.
- **Contraste** (calculé, norme WCAG, seuil de 4,5:1 pour du texte courant) : texte principal 13,0:1 ; titres 12,8:1 ; texte secondaire 5,5:1 sur l'ivoire (5,0:1 sur la barre latérale) ; texte cuivre 5,3:1 ; badge « crise manquée » 5,7:1 ; bouton principal 8,1:1.

*Une vérification a conduit à corriger la palette : le gris ardoise d'origine (4,4:1), le surtitre cuivre (3,3:1) et le badge corail (4,4:1) étaient sous le seuil et ont été assombris.*

*Point d'attention : la **phase post-critique** n'existe pas dans les données (les annotations ne donnent que le début et la fin de chaque crise). Sa couleur est prévue dans la palette mais **elle n'est pas affichée** : l'interface le rappelle plutôt que d'inventer cette phase.*

## C.4 La navigation et l'état

### La navigation par étapes

- **Quoi** : une barre d'étapes en haut de page (« Vue d'ensemble, 1 Prétraitement, ... 6 Prévention »). L'étape courante est soulignée, les étapes déjà consultées sont cochées, et chaque page se termine par « Précédente : ... / Suivante : ... ». **Pourquoi** : l'utilisateur voit à tout moment où il est dans le pipeline et quelle est la prochaine étape ; la liste de boutons radio de la barre latérale ne montrait pas cet ordre.
- **Comment** : chaque étape est un `st.button` dont le rappel `on_click` écrit l'étape dans `session_state["step"]` ; l'étape courante est le bouton de type `primary`. Un petit repère HTML invisible placé juste avant la rangée de boutons permet au CSS de ne cibler **que** cette rangée.
- **Pourquoi pas** `st.tabs` : Streamlit exécute le contenu de **tous** les onglets à chaque affichage, ce qui relancerait les calculs lourds de toutes les étapes. Avec des boutons, seule l'étape affichée s'exécute.

### La barre latérale : le contexte d'analyse

- **Enregistrement** : liste déroulante (chaque entrée affiche le nombre de crises, ex. « 210914B_A · 11 crise(s) »), suivie de chiffres clés (durée, canaux EEG, crises, fenêtres, part en crise).
- **Segmentation** : fenêtre (1, 2 ou 4 s), recouvrement (0, 50 ou 75 %), zone pré-crise (5 à 30 s) ; **Modèles** : taille minimale des feuilles du Random Forest (2 ou 5). Valeurs **discrètes** exprès : chaque variante du dataset coûte environ 1 minute de calcul et un fichier.
- Ces réglages sont **globaux** : ils changent le dataset lui-même, donc ils s'appliquent à toutes les étapes. Le rappel « Outil de recherche, pas un diagnostic médical » est affiché en bas du panneau et en pied de chaque page.

### La conservation des réglages entre les étapes

21 réglages d'étapes (passe-haut, K, seuils, modèle choisi, etc.) ont une valeur par défaut déclarée dans un dictionnaire `DEFAULTS`. À chaque exécution, chaque valeur est **réaffectée à elle-même** dans `session_state` : c'est ce qui empêche Streamlit de l'oublier quand l'étape n'est pas affichée. Les widgets concernés n'ont **pas** de paramètre `value=` (sinon Streamlit affiche un avertissement jaune). Résultat : on règle K à 20 à l'étape 3, on passe par la classification, et K vaut toujours 20 au retour. Le sélecteur de canaux de la page « Fenêtre par fenêtre » est réinitialisé (tous les canaux) uniquement quand le jeu de canaux change d'un enregistrement à l'autre (19 canaux contre 8 pour 200625A_F).

## C.5 Comment l'interface se connecte au modèle

Point important à bien comprendre : **il n'y a pas de fichier de modèle sauvegardé** (pas de `.pkl`). Les modèles sont entraînés **à la demande**, par le code de `src/models.py`, et le résultat est mis en cache.

| Étape | Ce qui se passe |
|---|---|
| 1 | Les réglages de la barre latérale (fenêtre, recouvrement, zone pré-crise) déterminent le dataset `ds` via `get_dataset`. |
| 2 | La page choisit K (nombre de features) : `best_feats(K, 0.9)` appelle `get_ranking` (classement) puis `select_uncorrelated` (suppression des redondances). |
| 3 | `get_loo(fenêtre, recouvrement, pré-crise, features, modèle, taille des feuilles)` appelle `loo_proba` : **22 entraînements**, chacun laisse un enregistrement de côté et prédit sa probabilité de crise pour chaque fenêtre. |
| 4 | La probabilité par fenêtre est utilisée par la page « Fenêtre par fenêtre » (verdict, courbe), la page « Détection temporelle » (seuil, lissage, fusion) et le tableau patient par patient. La petite fonction `loo()` mémorise aussi la configuration calculée, pour l'« état de l'analyse » de la vue d'ensemble. |
| 5 | À part, le bouton « Entraîner DT / RF / KNN / SVM » de la page Classification entraîne les 4 modèles sur un split 80/20 (`evaluate_all`) et garde le résultat dans `session_state`. |

**Pourquoi le leave-one-patient-out pour l'affichage**

- La probabilité affichée pour un enregistrement vient toujours d'un modèle qui **ne l'a jamais vu** : l'interface montre un comportement honnête, pas une mémorisation.
- Les 22 entraînements se font **une fois** par combinaison (features, modèle) : environ 1 min pour le Random Forest, davantage pour le SVM, puis instantané. Changer K ou de modèle déclenche un nouveau calcul ; revenir à une combinaison déjà vue est immédiat.

*Points d'attention : (1) pour **prédire sur un nouveau patient** en usage réel, il faudrait entraîner un modèle final sur tous les enregistrements et le sauvegarder (`joblib.dump`) : ce n'est pas fait. (2) L'application et `python -m src.evaluate` utilisent la **même fabrique de modèles** (`build_model`) : leurs chiffres sont comparables. Les valeurs de référence sont écrites dans `results/`, et la vue d'ensemble les affiche.*

## C.6 Les 7 pages, une par une

Chaque page suit la même trame : un en-tête (étape, titre, phrase d'explication, bandeau rappelant l'enregistrement et la segmentation actifs), un **panneau de paramètres**, puis les résultats organisés en sections.

**Vue d'ensemble**

- **Ce qu'on voit** : 4 compteurs (patients, crises annotées, fenêtres, % en crise) ; la chaîne de traitement en 6 étapes ; l'**état de l'analyse** (Fait / À faire pour chaque étape, d'après ce qui a réellement été calculé ou consulté dans la session) ; un accès rapide aux étapes ; les **résultats principaux** lus dans `results/summary_models.csv` et `results/summary_detection.csv` ; le tableau par enregistrement et le nombre de crises par patient.
- **Pourquoi** : donner d'emblée le contexte et les résultats disponibles. Aucune statistique n'est inventée : tout vient des données ou des fichiers de résultats (si `results/` est absent, un message indique la commande à lancer).

**1 · Prétraitement**

- **Ce qu'on voit** : un panneau (passe-haut, passe-bas, normalisation robuste, nombre de canaux), un choix de canaux et de plage de temps, puis 4 onglets : (a) trois courbes **brut** → **filtré** → **normalisé** par canal avec les crises annotées ; (b) spectre avant / après filtrage ; (c) spectrogramme ; (d) tableau de qualité par canal.
- **Comment** : `get_prep(REC, lo, hi)` renvoie les trois versions du signal. Le spectre utilise la méthode de **Welch** (moyenne de spectres de fenêtres de 4 s : estimation plus stable qu'une FFT unique) ; le spectrogramme utilise des tranches de 2 s décalées de 1 s, en décibels, limité à 30 Hz. La plage de l'axe du temps est fixée à la plage choisie.
- **Pourquoi** : *voir* l'effet de chaque étape de nettoyage. Les crises apparaissent dans le spectrogramme comme des bandes d'énergie vers 3 Hz.

**2 · Fenêtre par fenêtre**

- **Ce qu'on voit** : un panneau (K, seuil de décision, modèle) ; la navigation (curseur « Fenêtre n° », boutons Précédente / Suivante / **Prochaine alerte** / **Aller à la crise choisie**) ; pour la fenêtre courante : temps, part en crise, classe réelle, probabilité du modèle et un **verdict écrit** (Crise détectée, Pas d'alerte, Fausse alarme, Crise manquée) ; un **schéma du découpage en fenêtres** (fenêtres voisines, courante en cuivre, paramètres actifs) ; le signal des canaux avec 6 s de contexte ; la probabilité sur tout l'enregistrement ; les meilleures features de la fenêtre en écart à la normale (ligne repère à 2) ; le spectre FFT d'un canal ; les puissances relatives par bande.

```
key = f"wi_{REC}_{WIN}_{OVL}"                       # une position mémorisée par patient et réglage
st.session_state.setdefault(key, 0)
def jump(v):
    st.session_state[key] = int(min(max(v, 0), n - 1))
nav[0].button("  Précédente", on_click=lambda: jump(st.session_state[key] - 1))
```

```
wi = st.slider("Fenêtre n°", 0, n - 1, key=key)    # le curseur lit/écrit la même clé
```

- **Pourquoi un** `on_click` : Streamlit interdit de modifier la valeur d'un widget **après** son affichage dans le script. Le rappel s'exécute **avant** la ré-exécution, donc le curseur s'affiche déjà à la bonne valeur.
- **Verdict** : vrai positif si classe réelle = crise et probabilité > seuil ; faux négatif si crise réelle sans alerte ; faux positif si alerte sans crise ; sinon vrai négatif. **Écart à la normale (z-score)** : (valeur − moyenne hors crise) / écart-type hors crise.
- **Fenêtre courante translucide** : elle ne masque pas l'annotation de référence dessous (la zone de recouvrement est plus foncée).

### 3 · Meilleures features

- **Ce qu'on voit** : un panneau (nombre de features retenues, seuil de corrélation maximal) ; les features retenues sous forme d'**étiquettes** ; le classement en deux onglets : **tableau complet** (avec barre de score et colonne « Retenue ») et **graphique des 20 premiers scores** (retenue / non retenue, même ordre que le tableau) ; puis 3 onglets de comportement : évolution dans le temps chez le patient, distribution par classe, corrélations.
- **Comment** : `get_ranking` (4 critères, en cache) puis `select_uncorrelated`. La liste retenue est stockée dans `session_state["best_feats"]`, ce qui alimente la page Prévention et l'état de l'analyse.

### 4 · Classification

- **Ce qu'on voit** : un panneau d'entraînement (features meilleures ou toutes, K, séparation par patient ou stratifiée, pondération des classes) et un bouton **Entraîner** ; les résultats (tableau **Balanced Accuracy** / Précision / Recall / F1 avec la meilleure valeur surlignée, 4 matrices de confusion, cases « Data augmentation des crises » et « Seuil appris sur le train », tableau de référence stratifié contre par-patient) ; un encadré « Comment lire les métriques » ; puis le **tableau patient par patient** (TP, FN, FP, Recall, Précision, F1) pour un modèle et un seuil choisis.
- **Pourquoi un bouton** : un entraînement prend de 10 secondes à plusieurs minutes ; sans bouton, il se lancerait à chaque clic sur un curseur. Les résultats sont gardés dans `session_state`. Avec « Pondérer les classes », le KNN voit 5 non-crises pour 1 crise (il n'a pas de pondération), k = 31.

### 5 · Détection temporelle

- **Ce qu'on voit** : un panneau de 6 réglages (K, modèle, seuil, lissage, durée minimale, recouvrement minimal IoU) ; le **bilan** (crises retrouvées / manquées, fausses alarmes, intervalles prédits, appariement strict) ; un graphique en **trois bandes** partageant l'axe du temps : (1) l'EEG d'un canal avec les zones pré-critiques et les crises annotées ; (2) deux couloirs séparés, **« Référence »** (annotations des experts) et **« Modèle »** (prédictions de l'algorithme) ; (3) la probabilité de crise avec le seuil ; un encadré **« À lire avant d'interpréter »** (une prédiction n'est pas un diagnostic ; une « fausse alarme » peut être une crise non annotée ; la phase post-critique n'est pas annotée) ; la précision des bornes et les tableaux d'intervalles.
- **Comment** : `get_loo` fournit la probabilité ; `predictions_to_intervals` applique seuil, lissage, fusion, durée minimale ; `match_intervals` apparie strictement prédictions et crises. Le signal est sous-échantillonné (`sig[::4]`) **uniquement pour l'affichage**. La zone pré-critique est dérivée des annotations et du réglage « zone pré-crise » ; ce n'est pas une prédiction.

### 6 · Prévention (préparation)

- **Ce qu'on voit** : un encadré **Étude de faisabilité** et ses résultats (AUC et Balanced Accuracy de la pré-crise contre le calme, AUC selon le délai avant la crise) ; le nombre de fenêtres inter-critiques / pré-crise / crise ; un graphique qui **aligne toutes les crises sur leur début (t = 0)** et trace l'évolution moyenne des features choisies (plage réglable, par défaut −60 s à +20 s, ligne de base = 20 premières secondes).
- **Pourquoi** : si une feature commence à monter **avant** t = 0, elle porte un signal précurseur exploitable. L'étude de faisabilité conclut à un signal faible : ce n'est pas un système de prévention.

## C.7 Les choix techniques et leurs raisons

| Choix | Alternative | Pourquoi |
|---|---|---|
| Streamlit | Notebook, Flask, Dash | Demandé ; 100 % Python ; interactivité sans écrire de JavaScript |
| Plotly + gabarit commun `eeg` | Matplotlib, réglages graphique par graphique | Graphiques **interactifs** (zoom, survol) ; un seul gabarit donne polices, grilles, couleurs et survol identiques partout |
| `Scattergl` (WebGL) | `Scatter` | Dessine des centaines de milliers de points sans ralentir le navigateur |
| Navigation par boutons + CSS ciblé | `st.tabs`, boutons radio | Seule l'étape affichée s'exécute ; l'étape courante et les étapes consultées sont visibles |
| Un seul niveau d'encadré (`panel`) | Cartes partout | Interface plus légère : les encadrés ne servent qu'aux paramètres |
| Messages `callout` écrits en HTML | `st.info` / `st.warning` | Titre explicite (« À lire avant d'interpréter ») et couleurs de la palette, au lieu des bleu / jaune / rouge par défaut |
| Verdict en mot + couleur | Couleur seule | L'information ne repose jamais uniquement sur la couleur |
| Couloirs « Référence » et « Modèle » | Bandes superposées | On distingue au premier regard ce que l'algorithme prédit de ce que les experts ont annoté |
| Cache à deux niveaux | Aucun cache | Sans cache, 1 à 2 minutes de calcul à chaque clic |
| `cache_resource` pour les signaux | `cache_data` | Évite de copier de gros tableaux à chaque rerun |
| Tableaux d'affichage en float32 | float64 | Divise la mémoire par deux ; les calculs du dataset restent en float64 |
| Calcul dans `src/`, style dans `ui/` | Tout dans `app.py` | Les trois couches (calcul, style, orchestration) se modifient indépendamment |

## C.8 Comment l'interface a été vérifiée

- **Aucune fonctionnalité perdue** : un script a recensé tous les widgets de l'ancienne et de la nouvelle interface (type, plage de valeurs, valeur par défaut) pour les 7 pages. Résultat : **aucun widget manquant**, mêmes valeurs par défaut, **mêmes valeurs de métriques** et mêmes formes de tableaux. Les seuls ajouts sont : le tableau de résultats de référence sur la vue d'ensemble, l'onglet « scores comparés » et les liens directs.
- **Comportement** : tests automatiques (`streamlit.testing`) de la navigation (étapes, accès rapide, pied de page), de la conservation des réglages entre étapes, du changement de patient (y compris l'enregistrement à 8 canaux), de l'entraînement des 4 modèles, du calcul patient par patient et de la détection. Aucune exception ni avertissement.
- **Rendu réel** : l'application a été ouverte dans Edge (Chromium) et capturée page par page à 1500, 900 et 480 pixels de large.

### Défauts trouvés et corrigés pendant la refonte

| Défaut | Correction |
|---|---|
| Page « Fenêtre par fenêtre » : les bandes de crises éloignées étiraient l'axe du temps, ce qui écrasait le signal (défaut déjà présent dans l'ancienne version) | Plage de l'axe fixée à la fenêtre et à son contexte de 6 s |
| La fenêtre courante masquait le début de l'annotation de référence | Fenêtre translucide |
| Titre et légende des graphiques se chevauchaient | Mise en page commune (titre en haut, légende sous le titre) |
| Barre d'étapes : libellés qui se recouvrent selon la largeur | Boutons à la largeur de leur texte, défilement horizontal si l'écran est étroit |
| Écran étroit : colonnes écrasées, panneau latéral ouvert recouvrant la page | Les colonnes passent à la ligne sous 1100 px ; panneau replié par défaut sur petit écran |
| Sélection de canaux périmée après un changement d'enregistrement | Réinitialisée quand le jeu de canaux change |
| Contrastes insuffisants (gris ardoise, surtitre cuivre, badge corail) | Couleurs assombries (voir C.3) |

## C.9 Limites de l'interface

- **Vérifiée sur Edge (Chromium) uniquement** : non testée sur Firefox ni Safari.
- Le CSS cible la structure HTML de Streamlit **1.32** (attributs `data-testid`, sélecteur `:has()`). Une mise à jour majeure de Streamlit pourrait demander d'ajuster `ui/theme.py`.
- Les polices se chargent depuis Google Fonts : sans connexion, des polices système les remplacent (l'allure change légèrement).
- Premier affichage lent : construction du dataset (environ 1 minute), puis 1 à 2 minutes pour la première prédiction leave-one-patient-out.
- Pas de modèle sauvegardé ni de prédiction sur un nouveau fichier EEG déposé par l'utilisateur.
- `app_questions.py` (version organisée par questions) n'a pas été redessinée : elle reprend seulement les couleurs de base du thème.
- Les variantes de fenêtre (1 s, 4 s) et de recouvrement sont proposées mais leurs performances n'ont pas été évaluées.

# D. Résultats clés (à retenir)

| Élément | Valeur |
|---|---|
| Données | 22 enregistrements, 9,2 h, 256 Hz, 82 crises annotées valides, durée moyenne 13,2 s |
| Dataset | 33 206 fenêtres de 2 s, 1 168 de crise (3,5 %), 56 features dont 15 retenues |
| Features les plus discriminantes | Puissance de la bande 3 Hz (signature pointe-onde), amplitude, longueur de courbe |
| Meilleur modèle (fenêtres) | Random Forest : AUC 0,94, PR-AUC 0,59, F1 0,64 ; **non distinguable** du KNN corrigé (PR-AUC 0,56) et du SVM (0,54) avec 19 patients ; Decision Tree nettement plus faible (0,50) |
| Détection (appariement strict) | Random Forest : 68 / 82 crises (83 %), 37 fausses alarmes sur 9,2 h (4 par heure) ; KNN : 77 / 82 (94 %) mais 11 fausses alarmes par heure |
| Précision des bornes | Erreur médiane de 0 s au début et à la fin ; erreur moyenne de 1 à 2 s |
| Effet de la fuite d'information | RF : F1 de 0,76 en split stratifié contre 0,59 en split par patient |
| Étiquettes | Probablement incomplètes pour 210208B_G (3 salves visibles, aucune annotée) et 210427B_C : à faire valider |

# E. Points d'attention, limites et améliorations

## E.1 À assumer avec confiance (limites connues)

- **Étiquettes probablement incomplètes** pour 2 enregistrements (voir B.2) : les fausses alarmes et la précision sont sous-estimées ; décision à prendre avec l'encadrant.
- **Peu de patients** : 19 avec des crises. Random Forest, KNN corrigé et SVM ne sont pas statistiquement distinguables.
- **« Patient » = enregistrement** : l'Excel suggère que 210427B_C et 210504B_C (tous deux garçon de 11 ans) pourraient être le même enfant. Non vérifié ; si c'est le cas, le leave-one-patient-out est un peu optimiste pour ce cas.
- **Hyperparamètres** : réglage exploratoire seulement (écarts non significatifs), pas de validation croisée imbriquée.
- **Sélection des features sur tous les patients** (légère fuite, identique pour tous les modèles).
- **Normalisation sur l'enregistrement entier**, non applicable telle quelle en temps réel ; **pas de traitement spécifique des artefacts**, ni des protocoles de provocation présents dans les enregistrements (hyperpnée « HPN », stimulation lumineuse « SLI ») : leur effet sur les fausses alarmes n'a pas été analysé.
- **Taille de fenêtre non comparée** expérimentalement ; **pas de notebook** livré ; **prévention : étude de faisabilité seulement (signal faible, voir ci-dessous)** ; **pas de modèle sauvegardé**.
- **Choix d'annotations à confirmer** : dernière ligne conservée pour les doublons ; crise de 6 heures écartée.

## E.2 Améliorations possibles (à présenter comme perspectives)

- Faire valider les étiquettes par un expert, puis relancer l'évaluation (`python -m src.evaluate`).
- Régler les hyperparamètres par validation croisée **imbriquée** par patient ; tester un jeu de features **sans amplitude absolue** (cas 230718A_A).
- Features **inter-canaux** (cohérence, synchronie : les absences sont généralisées) et de **contexte** (fenêtres voisines) ; réseau convolutif 1D sur le signal brut.
- Sauvegarder un modèle final et permettre de déposer un nouveau fichier EEG dans l'interface.
- **Prévention** : l'étude de faisabilité (`python -m tools.prevention_study`) donne un signal faible (AUC ~0,64, Balanced Accuracy ~0,55, 17 patients sur 19 au-dessus du hasard). Pistes : plus de patients, suivi sur plusieurs fenêtres consécutives, modèles par patient, ECG.

## E.3 Questions difficiles et réponses solides

| Question probable | Réponse |
|---|---|
| Pourquoi pas l'Accuracy ? | Avec 96,5 % de fenêtres sans crise, un modèle qui répond toujours « pas de crise » a 96,5 % d'Accuracy et ne détecte rien. On affiche la **Balanced Accuracy** (0,50 pour ce modèle) et on suit le Recall (une crise manquée est le pire cas) contrôlé par le F1 et la PR-AUC. |
| À quoi sert la data augmentation ? | À montrer plus de crises au modèle pendant l'entraînement (copies bruitées des vraies crises, test jamais augmenté). Gain modeste, surtout pour le SVM et le Decision Tree, et en partie équivalent à un meilleur choix de seuil : je le présente comme une tendance, pas comme une preuve. |
| Pourquoi un seuil appris plutôt que 0,5 ? | Les probabilités des modèles ne sont pas calibrées de la même façon : à 0,5 chaque modèle est à un point de fonctionnement différent. Un seuil choisi sur une partie du train (jamais sur le test) compare les modèles à armes égales ; ils deviennent alors proches. |
| Pourquoi ces 4 modèles ? | Ce sont ceux du sujet. Ils couvrent 4 familles : règles (arbre), ensemble d'arbres (forêt), proximité (KNN) et frontière à marge (SVM). |
| Votre test est-il honnête ? | On a deux protocoles : le split stratifié du sujet, optimiste (fuite entre fenêtres chevauchantes), et le leave-one-patient-out, réaliste. Le F1 passe de 0,76 à 0,59 pour le RF : on a mesuré la fuite. Et la détection est évaluée avec un appariement strict : la mesure large précédente était optimiste de 7 points. |
| Quel est le meilleur modèle ? | Le Random Forest a la meilleure PR-AUC (0,59), mais il n'est pas statistiquement distinguable du KNN corrigé ni du SVM avec 19 patients ; seul le Decision Tree est nettement plus faible. Le choix dépend de l'objectif : RF pour peu de fausses alarmes, KNN corrigé pour ne rien rater. |
| Pourquoi avoir changé le KNN ? | Le KNN n'a pas de pondération de classes, donc il favorisait la classe majoritaire. En lui montrant 5 non-crises pour 1 crise et k=31, sa PR-AUC passe de 0,50 à 0,56. |
| Pourquoi 2 s et 50 % ? | 6 cycles d'un rythme à 3 Hz, résolution 0,5 Hz, une décision par seconde. C'est aussi la suggestion du sujet. Je n'ai pas comparé d'autres tailles, c'est une limite. |
| Pourquoi normaliser par médiane / IQR ? | Les amplitudes varient entre enfants ; médiane et IQR sont insensibles aux artefacts et aux crises, contrairement à la moyenne et à l'écart-type. |
| Pourquoi une bande à 3 Hz ? | Les pointes-ondes d'absence ont leur énergie vers 3 Hz. Les features de cette bande sont arrivées en tête du classement : l'hypothèse est confirmée par les données. |
| Vos étiquettes sont-elles fiables ? | L'alignement des annotations est validé (17 marqueurs sur 23 à ± 3 s). Mais 210208B_G n'a aucune crise annotée alors que 3 salves sont visibles : étiquettes probablement incomplètes, que j'ai signalées sans les modifier ; décision à prendre avec l'encadrant. |
| Pourquoi le RF échoue sur 230718A_A ? | Ses crises sont de faible amplitude mais très riches en 3 Hz ; le RF s'appuie sur l'amplitude absolue. J'ai testé une normalisation par enregistrement : elle dégrade les résultats globaux, donc abandonnée. Piste suivante : des features sans amplitude absolue. |
| Comment gérez-vous le déséquilibre ? | Pondération des classes pour DT, RF et SVM, sous-échantillonnage pour le KNN, métriques adaptées (Recall, F1, PR-AUC) et seuil réglable dans l'interface. |
| L'interface utilise-t-elle un modèle enregistré ? | Non : elle entraîne à la demande en leave-one-patient-out et met en cache, avec les mêmes modèles que le script d'évaluation. Pour un usage réel, il faudrait sauvegarder un modèle final. |
| Que fait la page Prévention ? | Une étude de faisabilité : on teste si un modèle reconnaît les 30 s avant une crise (contre des fenêtres calmes) sur un enregistrement jamais vu. Résultat : AUC ~0,64, Balanced Accuracy ~0,55 ; mieux que le hasard chez 17 patients sur 19, mais trop faible pour une alerte fiable, et le signal ne grandit pas à l'approche de la crise. On le présente comme une piste, pas comme un système. |

# F. Glossaire et commandes

## F.1 Glossaire

| Terme | Définition simple |
|---|---|
| EEG | Électroencéphalogramme : enregistrement de l'activité électrique du cerveau par des électrodes sur le crâne. |
| Crise d'absence | Coupure brève de la conscience (quelques secondes), surtout chez l'enfant ; sur l'EEG, rythme régulier d'environ 3 Hz. |
| Hz (hertz) | Nombre de répétitions par seconde. 256 Hz = 256 mesures par seconde ; un rythme à 3 Hz bat 3 fois par seconde. |
| Canal | La courbe d'une électrode (Fp1, Cz, O1...). |
| Fenêtre | Morceau du signal de durée fixe (ici 2 s) analysé d'un coup. |
| Recouvrement | Part commune entre deux fenêtres successives (50 % : décalage de 1 s pour une fenêtre de 2 s). |
| Feature | Nombre qui résume une fenêtre (amplitude, puissance à 3 Hz...). |
| FFT / spectre | Décomposition d'un signal en fréquences : quelle énergie à quelle fréquence. |
| Filtre passe-bande | Ne laisse passer que les fréquences d'une plage (ici 0,5-40 Hz). |
| Classe 0 / 1 | 0 = hors crise, 1 = crise. |
| Déséquilibre | Une classe est bien plus rare que l'autre (3,5 % de crises). |
| Train / test | Données pour apprendre / données jamais vues pour évaluer. |
| Fuite d'information | Quand le test contient des données trop proches de l'entraînement : le score paraît meilleur qu'il n'est. |
| Leave-one-patient-out | On entraîne sur tous les enregistrements sauf un, on teste sur celui-là, et on répète pour chacun. |
| Surapprentissage | Le modèle apprend les exemples par cœur et généralise mal. |
| Matrice de confusion | Tableau comptant TP, TN, FP, FN. |
| Bootstrap | Rééchantillonnage avec remise pour estimer l'incertitude d'une mesure (intervalle de confiance). |
| IoU (intersection sur union) | Mesure de recouvrement entre deux intervalles : durée commune divisée par durée de leur réunion (1 = identiques, 0 = disjoints). |
| Appariement strict | Chaque intervalle prédit ne peut correspondre qu'à UNE crise réelle (et inversement) ; les autres sont des crises manquées ou des fausses alarmes. |
| Pré-critique / critique / post-critique | Phases autour d'une crise : avant, pendant, après. Les données ne contiennent que le début et la fin des crises : la phase post-critique n'est pas annotée. |
| Cache | Mémorisation d'un résultat pour ne pas le recalculer. |

## F.2 Commandes utiles

```
pip install -r requirements.txt
python -m src.convert_to_csv         # une seule fois : .mat + Excel -> data_csv/ (vérifie l'égalité exacte)
python -m src.evaluate               # évaluation de référence -> results/ (environ 25 min)
python -m src.evaluate --augment     # même chose avec data augmentation (fichiers *_augment)
python -m tools.eval_stratified      # protocole du sujet : stratifié 80/20 répété (5 graines)
python -m tools.prevention_study     # étude de faisabilité de la prévention
streamlit run app.py                 # interface patient par patient (navigation par étapes)
streamlit run app_questions.py       # version organisée par questions du sujet
```

Le dépôt GitHub contient le code, le `README` et ce document ; les données (`.mat`, `data_csv/`, Excel) en sont volontairement exclues (données médicales et fichiers trop gros).